



DA610 - Advanced Queries and Data Processing

Student: Firstname Lastname
Version: 1.0

Introduction	1
Setup	1
MongoDB and Querying	2
Query Language Concept	2
Edge cases of selection and projection	3
Using \$elemMatch	5
Using Expressive Queries	7
Aggregation Pipelines	9
Pipelines	9
Simple Stages	9
Simple Expressions	10
\$project versus \$set versus \$redact	10
Summarization Stages	12
Using \$group	13
Using \$bucket	13
Using \$bucketAuto	14
Connection Stages	15
\$lookup	15
Expressive \$lookup	17
Output Stages	18
\$out and \$merge	18
Building a materialized view using \$merge	19
Management Stages	22
Blocking versus Non Blocking stages	23
List Comprehension Expressions	24
\$map	24
\$reduce	26
\$zip	26
\$range	26
Example: Determine if a Number is Prime	26
For the curious - Here is the version without \$reduce (not to be coded)	27
Generating Queries and Pipelines in Code	28
Using \$let	31
Updates with Aggregation Pipelines	32
Appendix	33

Introduction

This Workshop is designed to give you a little more depth with aggregation than the standard introduction does, it covers a wider range of concepts without being simply a list of stages to learn. Although it assumes you are not completely new to aggregation you should still be able to follow even if you are.

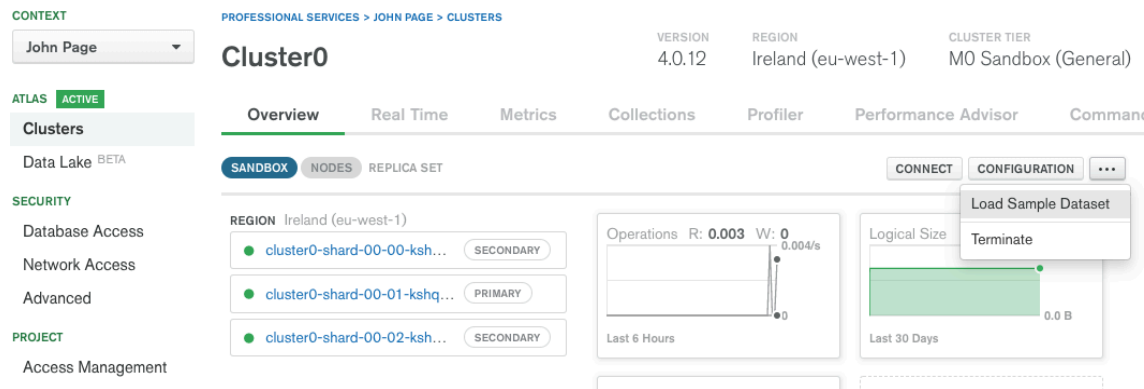
One thing you may note during this workshop is that it does not use the Aggregation builder in Compass or Atlas as much as you might expect. Whilst this is a nice way to visualize aggregations when writing them it does limit the way you think about aggregations so we focus on how to use the shell - or rather how to write aggregations in your code with the MongoDB shell being a proxy for a full programming language and driver.

You are expected to run all the examples and really understand them before moving on.

Setup

This workshop is not about measuring absolute performance nor does it use a very large quantity of data - you can therefore use either Atlas or your Laptop to host MongoDB. On your laptop this can be a Standalone or, as we normally recommend a single node replica set.

If you use Atlas you can load in the same data using this menu item.



If not using Atlas then you can load it using the following command into any MongoDB of your choice (4.2 or higher)

```
curl -OL https://mdb-tradecraft.s3-eu-west-1.amazonaws.com/m21_dump.tar.gz
tar xvzf m21_dump.tar.gz
mongorestore
```



MongoDB and Querying

Query Language Concept

We sometimes talk about MQL - the MongoDB query language, but arguably MongoDB does not have a query language, at least not in the way that SQL or SPARQL or Gremlin is a query language. The Wikipedia page on [Query Languages](#) doesn't mention MongoDB or MQL despite having a long list of query languages, including Couchbase's N1QL.

A query language, according to Wikipedia is defined thus: "Query languages or data query languages (DQLs) are [computer languages](#) used to make queries in [databases](#) and [information systems](#)."

But MongoDB does not have a language for this - we instead use a model of specifying - by objects - what data is required and leave the choice of language to generate these objects up to the developer. This is a very important distinction as it helps us understand that MongoDB Queries are intended to be written by code - not by a human.

The whole idea of a query being written in a human friendly form and then parsed is absurd in a day and age when 99.9999% of database queries are being auto generated from input data - the database user is no longer expected to write their own queries to interact with the computer (and hasn't been since the 1970s).

As you work through this - bear in mind that the job of the developer is not to 'write a query' or 'write an aggregation' but instead to construct the correct object(s) to inform MongoDB what information they need. It's a subtle but important distinction.

Edge cases of selection and projection

Before we begin with Aggregation it's worth reviewing some of the less well known parts of non aggregate queries. Essentially without aggregation MongoDB allows you to filter by row, to filter by field and to order results with a simple find(). In general these options are really easy to understand and are some of the first things people learn about MongoDB - then you can specify three objects to find.

- filter
- projection
- sort



which are used in a find operation and get back only the data you want. Let's try that with a basic query - query the supplied data and fetch back just the following. Put your query in the box below - use the MongoDB shell for this.

Write a query that returns from the sample_mflix database and the movies collection what movies released in 1933 won the most awards. Display only the title and awards text. Put your Query and answer in the box below.

```
db.movies.find({year:1933},{_id:0,"title":1,"awards.text":1}).sort({"awards.wins":-1}).limit(1)
```

You can also add `.next().awards.text` to the above query, to just get awards text.



This is a good time to talk about the process - it's likely when you filled in the box above you put a single line and if you got it correct it's approximately 175 characters long , it's also possible you missed the plural in movies as the correct answer returns three documents. This is the wrong way to think about query construction - it's not how you would do it in code and it's not a good way to do it even in the Mongo shell. Use your query to parameterise and run the code below

```
db = db.getSiblingDB("sample_mflix")

query = 
projection = 
sortOrder = 

doc = db.movies.find(query,projection).sort(sortOrder).limit(1).toArray()[0]
nwins = doc.awards.wins

query2 = 
projection2 = 
db.movies.find(query2,projection2)
```

Doing this is more readable, and when working in the shell or in an interpreted language it's easier to debug or see when you need brackets to be matched up. This code method is how you should always think about writing mongodb queries - not as JSON but as objects. In the rest of this workshop we are looking for sample answers like the one above with lines as short as possible



Using \$elemMatch

Some features of MongoDB are taught early on but little emphasis is given to them and the examples are very simplistic. \$elemMatch is one such feature

The \$elemMatch operator is used to query against arrays - and it's often thought of as a scoping operator - the array contains an element which matches this. When we are first shown it it's nearly always in an example like this.

```
r = {
  people : [
    { firstname: "john", lastname: "wayne" },
    { firstname: "jayne" , lastname: "doe" }
  ]
}
db.a.drop()
db.a.insertOne(r)

//Incorrectly finds the record - we don't have a "John Doe"
db.a.find({"people.firstname":"john","people.lastname":"doe"})

//Does what we expect - and doesn't find it
db.a.find({people : { $elemMatch : { firstname: "john", lastname:"doe"}}})
```

We need to come back round to the definition though - \$elemMatch finds the first array element where the supplied expression evaluates to true. We think of a MongoDB query as a 'query by example' in the simplest sense - because we can write a query like

```
{ country : "Kenya", age: 25 }
```

But this is actually a shortcut , a simplified form of the real query which reads

```
{ country : { $eq : "Kenya" } , age: { $eq : 25 } }
```

Although that's ugly for the simplest case, it helps us to understand that the argument to \$elemMatch can be any query expression. For a simple array of scalars we don't need \$elemMatch as find on an array will work on the single value. For an array of Documents we can use \$lt, \$gt, and anything else.

For arrays in arrays, we need to take a different approach.

If we want to find any sub-array that contains a certain value one might naively consider the following query would work



```
//Imagine we recorded the mean temperature of each week over the three
//months of summer. We have one of these records for each of our weather
//stations. A 3 x 4 array of values representing the 12 weeks

r = [{ counts : [ [ 25,17,29,28], [ 26,18,22,28], [ 24,23,20,19] ]},
      { counts : [ [ 15,17,29,22], [ 22,29,22,25], [ 25,27,33,25] ]}]
db.a.drop()
db.a.insertMany(r)

// find all documents with stations where the temperature average was 33
db.a.find({"counts": 33})
//Zero documents
```

Zero documents are found because there is no element within the array that is 33, there are only sub-arrays. We could try the positional operator, but once again, not what we want because the sub-array is not equal to 33. We would need to provide the whole sub-array as the query value:

```
// find all documents with stations where the temperature average was 33
db.a.find({"counts.2": 33})
//Zero documents
db.a.find({"counts.2": [ 25,27,33,25]}).pretty()
{
  "_id" : ObjectId("616cd83e78aac9859d5ee9c6"),
  "counts" : [
    [
      15,
      17,
      29,
      22
    ],
    [
      22,
      29,
      22,
      25
    ],
    [
      25,
      27,
      33,
      25
    ]
  ]
}
```

We also do not know which sub-array to target, so this is not an option either. We could revert to an anti-pattern of using unwind and group, but we really don't want to take this approach.

This is where the other use case of \$elemMatch comes into play. We can use \$elemMatch as it itself is an expression, and we can use it more than once, depending on the level of nesting. We will be using \$elemMatch in this case to perform the “find an element within my array where a



sub-element matches 33". Note this is a two step process and remember \$elemMatch needs an object as a value!

Write the query to find records (from the ones above) where there was a week with an average temperature of 33 - test by seeing it does not come back with any results when you search for a value of greater than 33.

```
query = { counts : { $elemMatch : { $elemMatch : { $gt : 33 }}} }  
db.a.find(query)
```

Now rewrite it to find where we had a week with an average temperature of 23 exactly

```
query = { counts : { $elemMatch : { $gt : 23 }}} }  
db.a.find(query)
```

When thinking about how to query something - especially if there are arrays involved - do not rule out \$elemMatch initially - it's more powerful than you expect.

Using Expressive Queries

From MongoDB 3.6 onwards, the query engine has access to aggregation expressions - we will talk about Aggregation expression later on in more detail but it is worth being aware that you can apply some basic computation to the values in a record, notably transforming or comparing them to each other. If the result of the computation is "true" then the record is returned. We can see this with a very simple query which returns everything.

```
db.movies.find({$expr : true})
```

When using \$expr - for each record it evaluates the value and returns the record or not - we can also use simple aggregation expressions like "\$fieldname" to get the value of a field

Run the following

```
db = db.getSiblingDB("sample_mflix")  
hasHighRating = {"imdb.rating": {$gt:9}}  
flagAsGood = {$set: { awesome : true}}  
  
db.movies.updateMany(hasHighRating, flagAsGood)
```

Now complete the following to find the movies with the awesome flag using \$expr. Do not use \$cond as part of it.


```
findAwesome = [REDACTED]  
db.movies.find(findAwesome)
```

What if you want to find something where the field isn't boolean - well you just need an aggregation expression that evaluates to boolean. `$cond` is an obvious candidate but `$isArray` and similar also return boolean. What's not at first obvious for some is that all the comparison operators and logical operators also return true or false, so `$cond` is not required. The real question is what can expressive queries do that you cannot do with a simple query. The answer is to compute values from the fields, and then use `$cond` to determine if the computed value meets some criteria.

```
condition = { $gt : [ "$imdb.rating", 9] }  
  
simpleQueryExpr = { $expr : condition}  
db.movies.find(simpleQueryExpr)  
  
redundantQueryExpr = { $expr : { $cond : { if: condition, then: true, else:  
false  }}}  
  
db.movies.find(redundantQueryExpr)
```

Interestingly and perhaps surprisingly the query

```
db.movies.find({"imdb.rating":{$gt:9}}).count()
```

and the logically identical query

```
db.movies.find({$expr : { $gt : ["$imdb.rating",9]}}).count()
```

return different results. This is due to the way `""` is compared to `9` - this is an important thing to note. MongoDB has semantics for ordering different types when sorting but apart from numbers does not compare different types, this is called type bracketing. In aggregation we do take the ordering of types into account so `null < 5 < ""` for example. In aggregation type bracketing does not happen. This can catch you out where you have nulls although treating `null < 0` may make sense.



Aggregation Pipelines

Pipelines

Aggregation pipelines allow you specify a number of stages, each of which reads a stream of incoming documents and outputs those documents transformed in some way, the stream may be shortened by filtering or grouping , reordered by a sort or lengthened via an unwind.

Simple Stages

We will assume you are already familiar with the basic concepts of a pipeline using \$match, \$sort, \$project - as you would in a find() as well as a simple example of \$group or \$unwind but let's test that with some basic exercises. You may need to refer to the online documentation to complete these.

In sample_mflix there is a movie collection. The countries field is an array listing what countries the movie was released in.

In our collection, each document contains an array of when that particular film was released in various countries. From all the documents in this collection, we want to determine when the first film was ever released in each country and order your results to show the latest year. { country: "Egypt", year:1960 }

```
unwind =   
group =   
project =   
sort =   
db.movies.aggregate([unwind,group,sort,project])
```

Simple Expressions

Aggregations also make extensive use of expressions. Expressions return one or more fields in the current document along with transformation of those fields - the simplest expression is "\$fieldname" which is just the content of the field - however there are many transformative expressions.

Expressions are written as { \$transformName: expression } or { \$transformName : [expression1, expression2, ...] } depending on whether it takes one or more than one input parameters. They always use an array for multiple 'arguments'



Some examples to try

```
r = { name: "John Page",
      awards: 22,
      merits : 5,
      startdate: ISODate("2013-08-02")
}

db.e.drop()
db.e.insertOne(r)

//$field

db.e.aggregate({ $project : { v : "$name" } })
db.e.aggregate({ $project : { v : { $substr: ["$name", 0, 1] } } })
db.e.aggregate({ $project : { v : { $split: ["$name", " "] } } })
db.e.aggregate({ $project : { v : { $sum : ["$awards", "$merits"] } } })
db.e.aggregate({ $project : { v : { $avg : ["$awards", "$merits"] } } })

//Number of milliseconds
now = new Date()
db.e.aggregate({ $project : { v : { $subtract : [now, "$startdate"] } } })
```

For further reference on aggregation expression operators check the documentation:
<https://www.mongodb.com/docs/manual/reference/mql/expressions/>

\$project versus \$set versus \$redact

Aggregation has four main ways to reshape a document, \$project, \$set, \$redact and \$replaceRoot

\$project is the earliest to be added and possibly the simplest to understand. You simply describe exactly what the documents should look like after this stage. It is however an absolute description rather than a relative one meaning as the fields in the source, or pipeline above, change you need to remember to carry them through the \$project stage. A \$project is not required as a performance/memory optimisation either as the pipeline can figure out for itself the minimum set of required data.

Write an aggregation for the sample_training.grades that only returns the elements within the 'scores' array where the 'type' is "exam"

```
use sample_training

match = { $match: { "scores.type": "exam" } }
projection = {
  $project: {
```

```

    exams_scores: {
      $filter: {
        input: "$scores",
        cond: {
          $eq: ["$$this.type", "exam"]
        }
      }
    }
  }
}

pipe = [match, projection]
db.grades.aggregate(pipe)

```

\$set by comparison is very useful, you can use it to add an individual, or multiple fields each of which is defined by an expression. Previously \$set was more commonly used with its alias \$addField, both still are available. You can also remove items via conditional expressions within a \$project or \$set by setting a field to \$\$REMOVE as part of the evaluated boolean expression. The \$unset stage is an alias and simpler format of the \$project stage that removes or excludes fields. Note that the behavior of \$unset in an update operation is different from \$unset in an aggregation pipeline.

Write an aggregation for sample_mflix which removes all mention of awards (even the fieldname) if Will Smith is in the cast

```

use sample_mflix
condition = 
pipe = [
db.movies.aggregate(pipe);

```

\$redact is a mysterious stage, little understood but designed to be able to traverse object structures as a hierarchy, it is seldom used and of very limited utility.

\$replaceRoot makes it easy to take any object you create from an expression and use it as the whole document in the next stage - there is no other way to do this.

These stages, especially \$set are where you will likely do the majority of computation and use the majority of CPU if you are making use of the aggregation pipeline as an in-database processing engine.



Summarization Stages

It is important to understand the overheads of using Aggregation, where the BSON data must be processed into internal objects by the server and converted back to BSON as results (except in a case where the whole pipeline can be expressed as a query).

Because of this - a pipeline that returns a large quantity of the data in the database or a large portion of the records touched may not make sense, it may be faster and simpler to simply return the raw data for processing at the driver end.

For aggregation to be truly useful it must be able to perform significant processing and aggregation of data at the server end, to return a summarized view of the information saving the cost in CPU and network of returning the information itself. We do this using the summarisation stages - ones which reduce documents to a smaller set of documents reducing the fidelity/resolution of the information.

`$group`, `$bucket`, `$bucketAuto`, `$sortByCount`, `$facet`

In short:

`$group` - create one output document for each unique value of a given expression in the input documents. Populate fields in that based on aggregate functions such as `$sum` and `$avg`. For example group cities by US state.

`$bucket` - create one output document for each supplied range of values, grouping together documents where a given expression falls into that range. For example, group people's cars by engine capacity rounded to the nearest 500. This can be done with `$group` but is less simple.

`$bucketAuto` - create N output documents of equal size by automatically determining what range of value to place in each group. This includes many variations on this to choose how these divisions are calculated.

`$sortByCount` - create a count of how many of each value we have for a given expression and return the counts, most common first.

`$facet` - Create a single document (Note the 16MB limit) containing the results of one or more pipelines based on the input data - for example using `$sortByCount` against multiple fields, each in a different sub pipeline. `$facet` does not run the sub pipelines in parallel and is only really useful where the preceding pipeline stages were expensive to compute.

Using the `sample_mflix` dataset, write pipelines to answer the following questions.

Count how many movies there are per Decade - for example 1890-1899, order the results by decade too. Do three ways using `$group`, `$bucket` and `$bucketAuto`. You will need to use expressions to do some data verification/cleansing as you do this - assume we ignore years with bad formatting.



Using \$group

```
db=db.getSiblingDB("sample_mflix")

fixyear =
groupbydecade =
sortbydecade =

db.movies.aggregate([fixyear,groupbydecade,sortbydecade])
```

Using \$bucket

To do this you should not have a hard coded list of boundaries but rather compute that dynamically in the shell to pass to the aggregation

```
db=db.getSiblingDB("sample_mflix")
db.movies.createIndex({year:1})
oldest =
db.movies.find({year:{$type:"number"}}).sort({year:1}).limit(1).next().year

latest=db.movies.find({year:{$type:"number"}}).sort({year:-1}).limit(1).next
().year

buckets=
fixyear =
intobuckets =

db.movies.aggregate([fixyear,intobuckets])
```

Using \$bucketAuto



Again you will need to dynamically compute the number of buckets needed. For this remember we are looking for by decade, not 10 equally sized groups

```
db=db.getSiblingDB("sample_mflix")
db.movies.createIndex({year:1})
oldest=db.movies.find({year:{$type:"number"}}).sort({year:1}).limit(1).next().year
latest=db.movies.find({year:{$type:"number"}}).sort({year:-1}).limit(1).next().year

numbuckets = 
print(numbuckets)

bucketauto = 
fixyear = 

db.movies.aggregate([fixyear,bucketauto])
```

At some point you might give up on the last one - that's the right thing to do as it's not possible, \$bucketAuto will give you equal sized groups not equal length groups.

Now try something harder, use \$group or similar to get the list of countries and their corresponding earliest release years and find the midpoint in that list. This is best done with a couple more stages and some expressions. The list of available expressions is here

<https://docs.mongodb.com/manual/meta/aggregation-quick-reference/#expressions>

The Output you are trying to get is

```
{ "country" : "Czech Republic", "year" : 1992 } or any other Country with 1992
```

```
//Like use but works in a .js file too
db = db.getSiblingDB("sample_mflix")
```



```
stage1 = [redacted]
stage2 = [redacted]
stage3 = [redacted]
stage4 = [redacted]
stage5 = [redacted]
stage6 = [redacted]
stage7 = [redacted]
db.movies.aggregate([stage1,stage2,stage3,stage4,stage5,stage6,stage7])
```

Connection Stages

\$lookup

During aggregation MongoDB can pull data into the pipeline from a second or subsequent collection. This is done using \$lookup - whilst this is equivalent to a left outer join (always include the row from the table on the left) it is important to consider the name \$lookup as that gives information about what it is intended to be used for. Document Schema design says you should embed information that is a direct child 1:1 or 1:N or any N:M lookup value that does not change very often. Only rapidly changing dimensions should be looked up.

\$lookup is in no way an excuse to use a Normalised rather than a Document style design and excessive use of \$lookup will slow things down in your app.

\$lookup has two modes - the simple version exactly matches a field in the source document to a field in one or more other documents and copies the contents of them in, it does this by running queries on the server to find matching documents. The more sophisticated version runs a different aggregation pipeline and includes the output of that as an array of documents in the source.

We will look at using both of them and things we need to optimise

Imagine we want to update our Movies database and create a new version where for each movie we can see all the other movies by the same director - to seed a 'you might also like' page. We don't want to group by director here; we literally want to \$lookup the director's other films and then embed some details of them.

Using a simple \$lookup, write an aggregation to do this. Add a field in each movie called seeAlso: and in it put the titles of the directors other movies. Use a \$set after the \$lookup to project just the fields you want.



```
db=db.getSiblingDB("sample_mflix")

lookupOthersByDirectors = {from: "movies",
                             }

movieTitlesOnly = 

db.movies.aggregate([lookupOthersByDirectors, movieTitlesOnly])
```

This is surprisingly slow.

Run the following to time it: we add a blocking stage (\$count) to ensure we have to finish the query, and not just retrieve the first few results, we also limit it to just the first 1000. If you get bored waiting, hit Control C and read the next paragraph. We could also have added .itcount() to the end instead which iterates the entire cursor adding network transfer times in to see the real performance.

```
var start=new Date()
db.movies.aggregate([{$limit:1000},lookupOthersByDirectors,movieTitlesOnly,{
  $count:"totaldocs"}])

var timeTaken=(new Date() - start)/1000
print(`took ${timeTaken} seconds`)
```

How long did it take?

That's too long , it normally takes about 20 seconds - can you fix it , why was it slow? - put the fix in the box below

Run it again - now how long does it take



Expressive \$lookup

We also have what are called expressive lookups, these are conceptually close to a nested select in SQL

A simple \$lookup is very limited - it can pull in whole documents based on a single exact matching field - it cannot summarise data or query on multiple fields - if we want to do more we need to use an expressive lookup.

Let us consider a scenario where we want to rank a director compared to all the rest - first we should work out a pipeline to rank a director based on the awards they have received. We also want to deal with the fact that movies have multiple directors which the previous \$lookup didn't handle.

Read through the following that computes the relative ranking of each director (not super accurately - that's a different exercise and has some edge cases). You cannot compute the rank of a Director without comparing to all the other directors so we can't just do this for one director with a match at the start.

```
unwindDirectors = { $unwind : "$directors" }
awardsPerDirector = { $group : { _id: "$directors",
awards: { $sum: "$awards.wins" } } }
orderByAwards = { $sort: { awards: -1 } }
groupToOneDoc = { $group: { _id: null, name: { $push: "$_id" } } }
unwindWithIndex = { $unwind: { path: "$name", includeArrayIndex: "directorRank" } }
dontStartAtZero = { $set: { _id: "$REMOVE",
directorRank: { $sum: [1, { $toDouble: "$directorRank" } ] } } }

db = db.getSiblingDB("sample_mflix")

dirRankPipeline = [unwindDirectors, awardsPerDirector, orderByAwards, groupToOneDoc,
unwindWithIndex, dontStartAtZero]

db.movies.aggregate(dirRankPipeline)
```

OK that's great, we can use it now to put the Director Ranking into each movie

```
//Have a filter to our pipeline - has to go at the end

justTheOne = { $match: { $expr: { $in: ["$name", "$$directors"] } } }
dirRankPipeline.push(justTheOne)

//Now we can parameterize it
addRank = { $lookup: { from: "movies", let: { directors: "$directors" },
pipeline: dirRankPipeline, as: "byRank" } }

db.movies.aggregate([addRank])
```



This is however amazingly slow if we apply it to the whole data set. Not because it is computing the rank of each director every time, aggregation is smart enough to cache lookup results where it can and we added a match at the end - but you are doing a linear lookup through an in memory set. Time how long it takes to compute the director ranking for 1,000 or 10,000 movies using \$limit - something that takes a notable time but not so you are waiting forever.

It takes X seconds to compute the ranked director score for N movies.

Output Stages

\$out and \$merge

The aggregation framework has the ability to redirect the output of an aggregation pipeline into a collection using \$out or \$merge stages.

Here the main differences between the two stages:

\$out	\$merge
Doesn't allow you to update an existing collection, the output collection will be replaced with the aggregation pipeline output. If the target collection already exists it will keep existing indexes	Allows to update existing collections, can be used for maintaining on demand materialized views
The output collection cannot be sharded	The output collection can be sharded

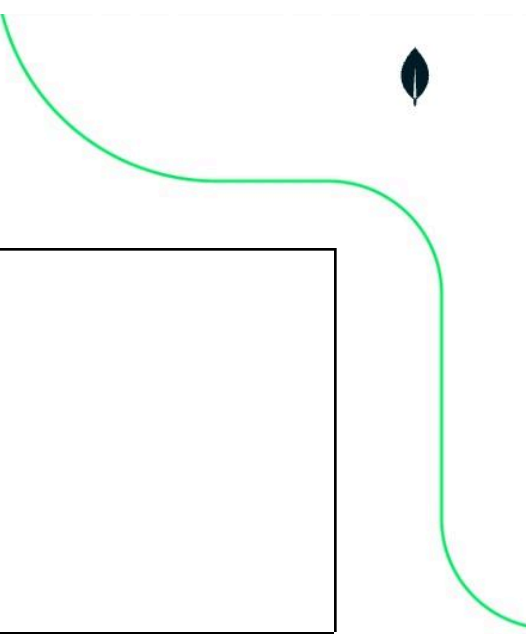
Considering the previous aggregation sub pipeline we built with director rankings, we can save the output using \$out as follows:

```
db.movies.aggregate([addRank, {$out: "rankDirector"}])
```

You will notice that this aggregation pipeline fails because not every document has the directors array, in order to make it work a simple fix would be to first consider only documents that have directors:

```
db.movies.aggregate([{$match: {directors: {$exists: true}}}, addRank, {$out: "rankDirector"}])
```

Now you have a new collection with the ranks in, can you change to a simple \$lookup to add the director ranking to each movie - put that aggregation in the box below and see how the speed compares to the expressive lookup version. Don't forget to add an index as needed.



```
db.rankDirector.createIndex([REDACTED])
```

```
db=db.getSiblingDB("sample_mflix")
```

```
lookupRankDirector = REDACTED
```

```
db.movies.aggregate([lookupRankDirector])
```

Building a materialized view using \$merge

Imagine you are collecting sensor values from an IoT device and you are storing this data in a collection. The raw data might have a too high resolution for showing it in a chart efficiently (MongoDB Charts for instance complains if it needs to render more than 5000 points). To solve this problem we are going to use \$merge to create an additional collection which aggregates the raw sensor data.

Open a mongo shell and run the following code to simulate a temperature and humidity sensor:

```

db = db.getSiblingDB("measurements");
db.sensorValues.createIndex({ count: 1 });
db.sensorValues.createIndex({ startTime: 1 });
var bucketSize = 10;

function simulateSensor() {
  while (true) {
    var temp = 10 + Math.random() * 20;
    var humidity = 10 + Math.random() * 90;
    db.sensorValues.updateOne(
      {
        count: { $lt: bucketSize }
      },
      {
        $push: { values: { "timestamp": new Date(), temp: temp,
humidity: humidity } },
        $min: { startTime: new Date(), tempMin: temp, humidityMin:
humidity },
        $max: { endTime: new Date(), tempMax: temp, humidityMax:
humidity },
        $inc: { count: 1 }
      },
      {
        upsert: true
      }
    )
    sleep(Math.floor(Math.random() * 200));
  }
}
simulateSensor();

```

This simulates an IoT device that is sending new sensor values in irregular intervals. The sensor values are stored in bucket documents, with a size of 10 by using an upsert update operation. We can even pre-aggregate a few things already in the update statement, like min and max values.

As the sensor data is too granular for us, we are now going to build an aggregation pipeline that will calculate the mean values for each bucket and incrementally update a collection called "aggregatedValues".

To prepare this put in the missing merge stage in the following script and then run it in a separate mongo shell:

```

db = db.getSiblingDB("measurements");
db.aggregatedValues.createIndex({ startTime: 1 });
var bucketSize = 10;

function updateMaterializedView() {
    //first check how much has already be done
    var lastProcessedStartTime = new Date(0);

    //get the timestamp of the last finished and processed bucket
    var c = db.aggregatedValues.find({ count: bucketSize })
        .sort({ startTime: -1 })
        .limit(1);

    if (c.hasNext()) {
        lastProcessedStartTime = c.next().startTime;
    }

    var startTimeFilter = { $match: {
        startTime: { $gt: lastProcessedStartTime }
    }
    };

    var unwind = { $unwind: "$values" }
    var group = {
        $group:
        {
            _id: "$_id",
            meanTemp: { $avg: "$values.temp" },
            meanHumidity: { $avg: "$values.humidity" },
            startTime: { $first: "$startTime" },
            endTime: { $first: "$endTime" },
            count: { $first: "$count" },
        }
    };

    // complete the merge stage
    var merge =
    [
        $merge: {
            into: "$_id",
            from: "$values",
            on: "$_id",
            as: "$values",
            whenMatched: "replace",
            whenNotMatched: "insert",
        }
    ];

    db.sensorValues.aggregate(
        [
            startTimeFilter,
            unwind,
            group,
            merge
        ]
    )
}

while(true) {
    updateMaterializedView();
}

```

```
print("db.aggregatedValues.count(): "+db.aggregatedValues.count());  
//update the view every 5 seconds  
sleep(5000);  
}
```

This script will now update the aggregatedValues collections every 5 seconds. Because we are only processing bucket documents which are either unfinished or for which there is no aggregated version in aggregatedValues yet, we only need to process a small number of documents during each iteration.

Management Stages

Several commands for collecting statistics and other useful information from the MongoDB server have an aggregation pipeline counterpart, some even only exist as an aggregation pipeline stage. This has the advantage that they can be distributed in a sharded cluster and it makes analysing the resulting data easier.

The following management stages are available:

- \$collStats
- \$currentOp
- \$indexStats
- \$listLocalSessions
- \$listSessions
- \$planCacheStats

A few ideas what you could do with them would be:

- Use them in a view and then visualize them with MongoDB Charts
- Build your own simple monitoring by writing the results into a collection using the \$merge stage and then visualize it in MongoDB Charts
- Build an aggregation pipeline to analyse the results of \$planCacheStats to see which Index is used by which query to identify Indexes that could be removed

These are not intended to be accessed by applications at runtime - they can have surprising levels of locking or CPU requirements and therefore should only really be used by an administrator.

Blocking versus Non Blocking stages

Aggregation stages can be divided into two categories - blocking, and non-blocking. A non blocking stage can immediately pass its input document to the next stage in the pipeline in a streaming fashion - for example \$set or \$match (which may either pass the document or remove it from the stream). \$unwind is also a non blocking stage

Blocking stages however need to see all documents before they can send any to the next stage - the blocking stages are \$sort, \$group, \$facet, \$bucket, \$bucketAuto and \$sortByCount.



In a sharded cluster - be aware that as soon as you use a blocking stage you cause all the results to be coalesced onto a single host - for example when sorting the results are sorted on the shards but then merged onto a single host.

For this reason always see if you can find a way to avoid blocking stages. For example, to sort an array inside a document you could use \$unwind, \$sort and \$group by _id with \$push - but this is exceptionally inefficient. \$group assumes the input may come from multiple documents and so has to wait for every document to complete.

You could instead use the \$set stage with the \$setUnion operator and use the empty set [] as the second argument for \$setUnion- MongoDB sorts to compute sets. This is not a documented behaviour and therefore could change in future but is the simplest option currently especially in an ad-hoc aggregation. There is also a longer more complex way which still doesn't use \$group but uses comprehension as below, we are not listing that here but it is possible to write a full sort function in aggregation.

List Comprehension Expressions

In the function updateMaterializedView() of the sensor example we used a combination \$unwind and \$group in order to calculate the average values in a bucket document. As \$group is a blocking stage and was used to aggregate something inside the same document, we can replace it with a non blocking \$project stage. This is why \$unwind followed by \$group is considered an anti pattern. Luckily there are many list comprehension expressions that allow us to process arrays directly in a projection stage without unwinding it.

Complete the \$project stage that could be used to replace the \$unwind and \$group stage in the updateMaterializedView() function of the materialized view example:

```
var aggregate = {
  $project: {
    [REDACTED]
    [REDACTED]
    [REDACTED]
  }
}
```

Besides the \$avg expression there is a whole list of other list comprehension expressions that we can use to work on arrays during projection stages, like \$map,\$reduce,\$zip and \$range.

\$map

The expression \$map is applied to every element in an array, each item is interacted with through \$\$this by default. The result will be an array of values each with the map actions applied. - Below



is an example that squares all the elements in an array called `some_list` and sets it in a field called `squaredlist`.

```
{ $set : { squaredlist :  
  { $map : {  
    input: "$some_list",  
    in: { $mul : [ "$$this", "$$this" ]  
  }  
}
```

If we go back to our `$elemMatch` example, we can use `$map` combined with `$filter` to only return the sub-arrays that satisfy our query

```

//Imagine we recorded the mean temperature of each week over the three
//months of summer. We have one of these records for each of our weather
//stations. A 3 x 4 array of values representing the 12 weeks

r = [{ counts : [ [ 25,17,29,28], [ 26,18,22,28], [ 24,23,20,19] ]},
      { counts : [ [ 15,17,29,22], [ 22,29,22,25], [ 25,27,33,25] ]}]
db.a.drop()
db.a.insertMany(r)

//find all documents that have a weekly average of less than 20 and only
//return the sub-arrays that match this query
match = {
  $match: {
    "counts" : {
      "$elemMatch" : {
        "$elemMatch" : {
          "$lt" : 20
        }
      }
    }
  }
}
project = {
  $project: {
    lows: {
      $filter: {
        input: "$counts",
        as: "topLevel",
        cond: {
          $in: [
            true,
            {
              $map: {
                input: "$$topLevel",
                as: "innerLevel",
                in: {$lt: ["$$innerLevel", 20]}
              }
            }
          ]
        }
      }
    }
  }
}
db.a.aggregate([match,project])

```

\$reduce

The expression \$reduce iterates over an array returning a single value, you specify the original value and the value after examining each list element. To sum a list

```
{ $set : { sumofvals :
  { $reduce : {
    input: "$some_list",
    initialValue: 0,
    in: { $sum : [ "$$value","$$this"]
  }
}
}
```

\$zip

The expression \$zip takes two or more lists and returns a list of lists where the first list has element zero for each of the inputs, the second list has element one of each of the inputs, and so forth. Such as: [1,2,3] [5,6,7] to [[1,5],[2,6],[3,7]]

\$range

The expression \$range creates a list/array of values from 1 to n. It can take a step value.

These four operations can essentially create a Universal Turing machine and, with effort, solve any problem - they are immensely powerful for computing things inside the database.

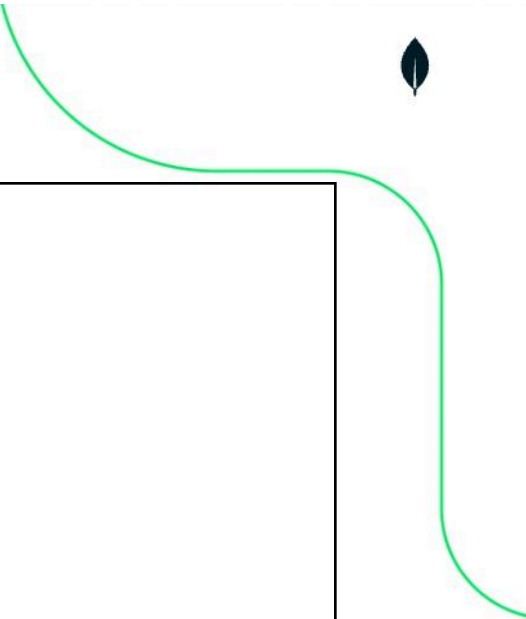
Example: Determine if a Number is Prime

Sometimes it is necessary to determine [if a number is prime](#) and there is no \$isPrime expression in Aggregation. You can use aggregation to implement an algorithm by taking advantage of list comprehension operators. In this exercise, we are going to use \$range, \$map, and \$reduce in conjunction with \$project to accomplish our goal.

First create a range of documents with values to test

```
db.numbers.drop()
for(x=0;x<1000;x++) {
  db.numbers.insertOne({_id:x})
}
```

Now write an aggregation to tell us which are prime (Ignore 0 and 1), use \$range to generate a list of factors, \$map to divide the number by each and \$reduce to determine if any are an exact division. It's possible to do this without the \$reduce as well.



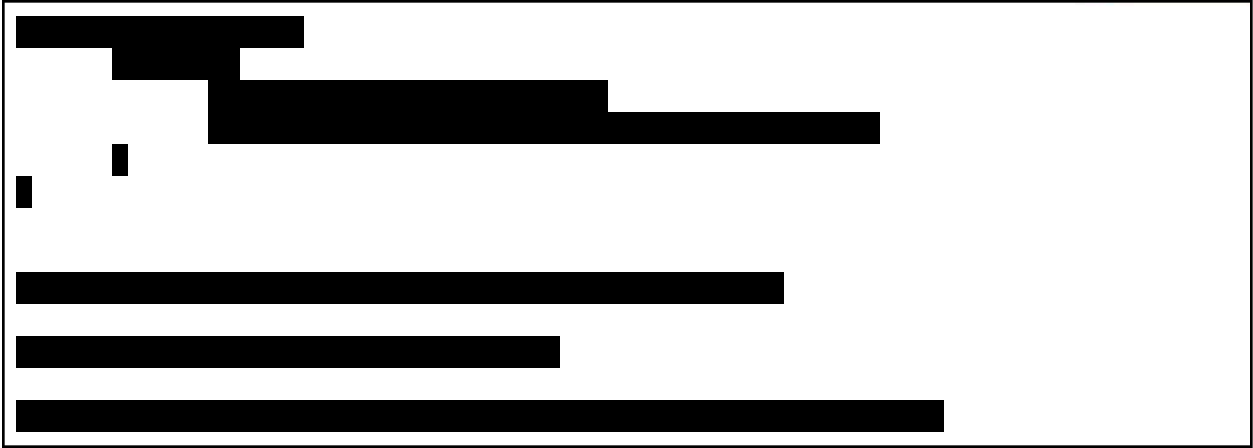
```
possible_factors =
[REDACTED]

isPrime =
[REDACTED]

db.numbers.aggregate([ { $set: { prime: isPrime } } ])
```

For the curious - Here is the version without \$reduce (not to be coded)

Age Group	Male (%)	Female (%)
18-24	~65%	~35%
25-34	~55%	~45%
35-44	~45%	~55%
45-54	~35%	~65%
55-64	~25%	~75%





Generating Queries and Pipelines in Code

Aggregation pipelines or even queries can quickly become very complex and would be really difficult to write completely by hand.

Luckily a big advantage of them compared to SQL queries is that they are objects, which makes it easy to write builder functions for them or to have variables with reusable parts of an aggregation pipeline.

Normally we are writing them in code - not as one-offs in the shell anyway

Here is a list of cases when it is especially useful to build generator functions:

- The aggregation pipeline is complex and it simplifies the code
- Parts of the aggregation pipeline can be reused.
- You want to use data to generate the pipeline
- You like clean and readable code

Here is a challenge. Imagine you have a collection which has temperatures as an object. Some are stored as { temp: x, unit: "c" } and some are {temp: x, unit: "f" }

When writing aggregations (or even queries) you want to query for either - write the function `fahrenheit` that makes the following code work. and finds either version. Remember whatever your language you are never manipulating Javascript strings.

```
db.temperatures.find({ temperature: fahrenheit(451) })
```

This is a really difficult concept to express as the jump from simple to requiring a function to produce can be large - it's important to bear in mind this concept of queries being generated and being generated from nests of functions

Here is a final example - this code returns an expression to XOR two integers - the shift and getbit functions can be easily reused to implement OR and AND. Walk through this with your instructor.

```

function binxor(a, b) {
  return {
    $reduce: {
      input: {
        $range: [31, -1, -1]
      },
      initialValue: 0,
      in: {
        $add: [{
          $multiply: ["$$value", 2]
        },
        {
          $cond: [{
            $eq: [1, {
              $add: [getbit(a, "$$this"),
                getbit(b, "$$this")
            ]
          }], 1, 0]
        }
      ]
    }
  ]
}
}

function getbit(integer, bit) {
  return {
    $mod: [shiftright(integer, bit), 2]
  }
}

//Simple version
function shiftright(integer, bits) {
  return {
    $floor: {
      $divide: [integer, {
        $pow: [2, bits]
      }]
    }
  }
}

```

Note: There is boolean XOR in the MongoDB aggregation pipeline, using [\\$bitXor](#) this example could be simplified.



Using \$let

The expression \$let is often misunderstood - it comes back to the old "Never recompute what you can precompute" mantra of code optimization. The following are equivalent. In this example \$a is different in each record so we cannot simply hard code the square root of a into our aggregation - technically we could add it as a temporary field however \$let solves this as a temporary variable.

```
values = { $range : [ 0, 360] }

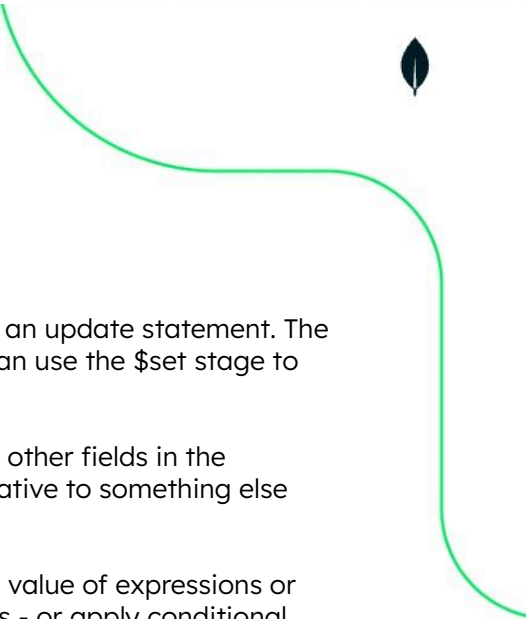
// $sqrt of $a computed 360 times

{
  $set : {
    normals : {
      $map : {
        input: values,
        in : {
          $divide : [ "$$this", {$sqrt : "$a"}]
        }
      }
    }
  }
}

// Compute $sqrt of a once and use in the rest as a variable

{
  $set : {
    normals : {
      $let : {
        vars : {
          sqrtofa : {
            $sqrt: "$a"
          }
        },
        in: {
          $map : {
            input: values,
            in : {
              $divide : [ "$$this", "$$sqrtofa" ]
            }
          }
        }
      }
    }
  }
}
```

Note - you cannot pass variables between documents or stages.



Updates with Aggregation Pipelines

Since MongoDB 4.2 it is also possible to use aggregation pipelines inside an update statement. The aggregation pipeline is then used instead of the update document and can use the \$set stage to set fields in the updated document.

The purpose of this is to be able to perform an update which depends on other fields in the document - before that only \$inc and \$mul were operations that were relative to something else (arguably \$push and \$addToSet)

Aggregations in updates are limited to setting and unsetting fields to the value of expressions or replacing the entire document. You can set one value to the sum of others - or apply conditional logic.

Write an update to add a flag in the numbers collection saying if the number is prime or not. Put the update command in the box below.

```
possible_factors = []
for i in range(2, int(n**0.5)+1):
    if n % i == 0:
        possible_factors.append(i)
        possible_factors.append(n//i)

isPrime = True
for i in possible_factors:
    if i != n:
        isPrime = False
        break

if isPrime:
    print(f"{n} is a prime number.")
else:
    print(f"{n} is not a prime number. Its factors are: {possible_factors}")
```

Appendix

Additional Reference

Aggregations book: <https://www.practical-mongodb-aggregations.com/>

How to find a prime number?

Problem:

We want to find if the given number N is prime.

Solution (pseudocode):

```
counter = 2
```

```
while counter < Square root of N  
begin
```

```
    if counter divides N without a remainder return false
```

```
    add 1 to counter
```

```
end
```

```
return true
```